AMTCS - IoT project on Automating Measurements Using a Temperature Controller and Spectrometer, Initially Simulated (client: Tyndall National Institute)

Project Proposal
Marina Zubialevich, 20108904
Higher Diploma in Science in Computer Science.

Project Type: Work-Based Project

Project Category: Hybrid – Physical computing/hardware + Server (optional) + Mobile App (for monitoring, optional)

Project Description:

The goal of this project is to develop software that automates temperature-resolved spectroscopic measurements in a research laboratory at the Tyndall National Institute. Currently, the stakeholder carries out these measurements manually. Although each individual spectrum takes only about one minute to acquire, the time required for the system to stabilise at the next temperature is approximately 30 minutes. This interval is too long to simply wait and too short to undertake other meaningful tasks. As a result, automating this workflow would significantly improve efficiency and save time.

The proposed solution is to implement a programmed logic controller (PLC), ideally running on a Raspberry Pi or alternatively on a PC. The controller will communicate with both the temperature controller and the Horiba spectrometer. It will monitor when the system reaches the target temperature and then automatically instruct the spectrometer to perform the measurement and store the acquired spectrum.

Optionally, the system may include a lightweight server (possibly outside the local lab network) that receives real-time updates from the PLC and provides this information to a mobile application. This would allow the user to remotely monitor measurement progress. Implementation of this feature will depend on Tyndall's security policies regarding network access.

Development will use the Horiba spectrometer SDK (SynerJY) and the ASCII-based command protocol of the temperature controller. The PLC software will be written in either Visual Basic or C++ (languages supported by SynerJY SDK). If implemented, the optional server will be developed in Python, and the companion mobile app in Kotlin or Java.

The first complete version of the system will be delivered as a single release. Any further improvements or additional features may be developed later, based on stakeholder feedback and practical usage.